リモートアクセス型マルウェアのファイル操作機能に起因する MFT 痕跡を用いた情報漏洩範囲特定手法の検討

関谷 信吾[1]　増田 利来[1]　中野 心太[1]　安田 真悟[2]

概要：サイバー攻撃に起因するインシデントでは，情報資産へのアクセスや持ち出しの有無を確認することが重要である．しかし，Windows OS には外部送信されたデータの内容を直接記録する標準機能がなく，情報漏えいの判断はファイル操作の痕跡や通信量に基づく間接的な推定に依存する．本研究では，複数のリモートアクセス型マルウェアを対象とした動的解析により，マルウェアのファイル操作機能を実行した際，参照したフォルダ配下のファイルの AccessTime が MFT 上で短時間に集中して更新される特徴を確認した．この痕跡は，マルウェアによって参照または操作された可能性のあるファイル群の抽出に有効であり，情報漏えい範囲を絞り込む精度の向上に寄与する可能性がある．

キーワード：インシデントレスポンス，デジタルフォレンジック，マルウェア解析

A Study on a Method for Identifying the Scope of Data Exfiltration Using MFT Artifacts Caused by File Operation Functions of Remote Access Malware

Shingo Sekiya[1]　Riku Masuda[1]　Shinta Nakano[1]　Shingo Yasuda[2]

Abstract: When an incident caused by a cyberattack occurs, it is important to determine whether information assets have been accessed or exfiltrated. However, Windows OS does not provide a function for directly recording the contents of data transmitted outside the system. Therefore, judgments regarding data exfiltration must rely on indirect estimation based on file operation artifacts and network traffic volume. In this study, through dynamic analysis of multiple remote access malware samples, we observed a characteristic behavior in which the AccessTime values in the MFT of files under a referenced folder were updated intensively within a short period when the malware's file operation function was executed. The results indicate that this artifact is effective for extracting groups of files that may have been referenced or operated on by malware, and may contribute to improving the accuracy of narrowing down the scope of data exfiltration.

Keywords: Incident Response, Digital Forensics, Malware Analysis

1. はじめに

サイバー攻撃に起因するインシデント発生時には，侵入経路，被害範囲，設置・実行されたファイル，ならびに情報漏えいの有無をデジタルフォレンジック等の技術を用いて明らかにすることが求められる．侵入経路や実行ファイルについては，ログやファイルシステム上の痕跡から一定程度特定できる場合が多い．一方で，情報資産が実際に参照または外部へ持ち出されたかについては，その事実を直接的に示す証跡が残らない場合も多く，調査上の課題となっている．特に Windows OS には，外部送信されたデータ内容を標準的に記録する機能がないため，情報漏えいの判断は，ファイル操作の痕跡や通信量等に基づく間接的な推定に依存せざるを得ない．

本研究では，リモートアクセス型マルウェアおよびサポート詐欺等で悪用されるリモートアクセスツールを対象に，これらが備えるファイル操作機能を実行した際に残存する痕跡を分析する．これにより，情報漏えい範囲の特定における絞り込み精度の向上に資する特徴的な痕跡を明らかにすることを目的とする．

2. 関連研究

NTFS のタイムスタンプに関しては，ファイル操作に伴う更新規則の解明や，タイムスタンプの改ざん・隠蔽の検知を目的とした研究が行われてきた．Chow ら[1]は，コピー・更新・削除といった各種操作と MAC times（更新・アクセス・作成時刻）の対応関係を経験則として整理し，イベント再構成への活用を示した．特に，多数のファイルのアクセス時刻が近接して観測される現象が，アンチウイル

1 株式会社日立システムズ ビジネスイノベーション統括本部
Lumada 推進本部 セキュリティイノベーション推進部
Hitachi Systems, Ltd. Security Innovation Promotion Department

2 国立研究開発法人情報通信研究機構　サイバーセキュリティ研究所
National Institute of Information and Communications Technology
Cybersecurity Research Institute

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

スによるスキャンやサムネイルプレビューなどの機械的な一括処理に起因することを指摘している．Galhuber ら[2]は，より新しい Windows 環境を対象に，正当なファイル操作におけるタイムスタンプの更新規則を体系的に整理するとともに，改ざんツールによる痕跡の検知についても検討している．また，Neuner ら[3]は，タイムスタンプのサブ秒領域を利用した情報隠蔽と，その時刻分布の統計的特徴に基づく改ざん検知手法を検討している．

これらの研究は，主として正当な操作におけるタイムスタンプ更新規則の解明や，改ざん・隠蔽の検知を目的とするものである．これに対して本研究は，リモートアクセス型マルウェアおよびリモートアクセスツールのファイル操作機能を実行した際に，参照されたフォルダ配下のファイルの AccessTime が短時間に集中して更新される挙動に着目し，これを攻撃者によるファイル探索範囲，ひいては情報漏えい範囲の推定に応用する点に特徴がある．すなわち，一括処理がアクセス時刻の近接を生じさせるという Chow らの知見を，インシデント調査における探索範囲の特定という課題に対して，マルウェアのファイル操作を対象に発展させるものと位置づけられる．

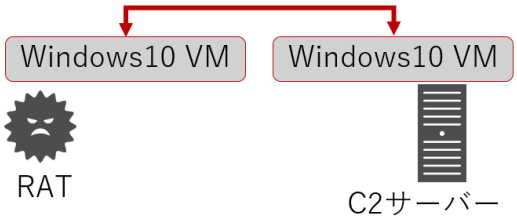


図 1　RAT による痕跡の検証環境
Figure1 An environment for verifying traces left behind by RAT.

3. RAT の各機能に対する痕跡の調査

インシデント調査では，端末が RAT（Remote Access Trojan）に感染した事実は特定できても，RAT および C2 サーバーが備える多数の命令のうち，実際にどの操作が実行されたかまで特定することは困難である．特に，探索または窃取されたフォルダおよびファイルの特定は，情報漏えい範囲の判断に直結する一方で，その事実を直接示す証跡が残らないことが多く，調査上の課題となっている．

一方，RAT の各機能の実行に伴い，Sysmon ログ，イベントログ，MFT，レジストリなどに，実行された命令を推測できる痕跡が残存する可能性があると考えた．そこで本章では，RAT の各機能を実行した際に，Windows 端末上に残存する痕跡を調査する．

3.1 検証方法および検証環境

Windows 10 仮想マシン 2 台に，それぞれ RAT と C2（コマンドアンドコントロール）サーバーを配置した．解析者

表 1　AsyncRAT に機能による痕跡の検証結果
Table1 Verification results regarding traces left by AsyncRAT functions

実行する機能	Sysmon ログ	イベントログ	MFT	レジストリなど実行痕跡
Zip ファイル解凍後検体設置	-	-	○	-
管理者権限で検体実行	○	-	○	○
ユーザー権限で実行	○	-	○	○
systeminfo コマンドをリモート実行	-	-	-	○
Execute .NET CODE 機能	○	-	-	○
ディスクでの exe 実行機能	○	-	○	○
メモリでの exe 実行機能	-	-	-	-
Password Recovery	○	-	-	-
Keylogger	○	-	-	-
File Manager 起動	○	-	-	-
File Manager から Zip ファイルをダウンロード	○	-	-	-
File Manager からファイル削除	-	-	○	-
File Manager からファイル作成	-	-	○	-
Remote Desktop 機能実行	○	-	-	-
Remote Desktop で Zip ファイル作成	-	-	○	-
リモートからのブラウザ起動	○	-	-	-
検体側端末の再起動	○	-	-	-
C2 サーバーに感染端末の情報送信	○	-	-	-
管理者権限の取得（GetAdmin 機能）	○	-	-	-
任意の画像ファイルで壁紙変更機能	○	-	-	-

© 2026 Information Processing Society of Japan

が C2 サーバーを操作し，ファイル操作を含む各機能の実行前後で特徴的な痕跡が残存するかを解析・検証した．痕跡の有効性については、以下の 2 点から評価する．

① RAT を介した操作時刻に痕跡が残存するか
② 探索した範囲が痕跡で特定可能か

本検証で使用した構成を図 1 に、環境情報を以下に示す．

- 環境：仮想マシン 2 台
- OS：Windows 10
- ネットワーク：C2 サーバーと RAT はプライベートネットワークで通信

検証対象は，C2 サーバーおよび RAT のビルダーが公開されているマルウェアから選定し，検証当時に多数観測されていた AsyncRAT および DCRat を採用した．

3.2 AsyncRAT における痕跡の検証と考察

解析者自身が C2 サーバーから AsyncRAT の各機能を操作し，操作を実施した時刻を基準として，操作によって残存した痕跡を調査した．

3.1 節に示した構成で痕跡を検証した結果を表 1 に示す．プログラム実行に関する操作では Sysmon ログおよびレジストリに，ファイル作成・削除に関する操作では MFT に痕跡が残存した．また，File Manager 機能でフォルダを操作した際には，操作していないファイルを含め，当該フォルダ内の全ファイルの AccessTime(MFT)がほぼ同時刻に更新された．

この結果から，File Manager 機能に伴う AccessTime の更新範囲は，参照されたフォルダを推定する手掛かりとなることが確認できた．

3.3 DCRat におけるファイル操作痕跡の検証と考察

AccessTime の挙動が AsyncRAT 固有のものか確認するため，他の RAT でも同様の結果が得られるという仮説のもと、DCRat を対象に検証した．

3.1 節に示した構成で検証した結果，DCRat においても AsyncRAT と同様に，File Manager 機能を介してフォルダを操作した際，当該フォルダ内の全ファイルの AccessTime(MFT)がほぼ同時刻に更新された．

このことから，同挙動は AsyncRAT 固有ではなく，複数の RAT に共通して生じる可能性がある．したがって，複数ファイルの AccessTime が操作時刻付近で集中して更新されている場合，ファイル探索または窃取範囲を推定する手掛かりとなると考えられる．

4. 原因となるコードの調査

前章の検証により，RAT の File Manager 機能の利用に伴う特徴的な AccessTime の更新挙動を確認した．しかし，この挙動が検証した検体固有の実装によるものかは明らかではなく，インシデントが発生するたびに，当該検体の動作検証を個別に実施する必要が生じる．また，ソースコードやビルダーが公開されている RAT は限られるため，検体によっては動作検証自体が困難であるという課題がある．

AsyncRAT および DCRat では同様の AccessTime 更新挙動が確認されたことから，特徴的な痕跡は RAT 固有ではなく，共通して利用されるメソッドや API に起因すると考えた．そこで，File Manager 機能で利用されるメソッドおよび Windows API を調査し，これらが AccessTime 更新の原因であるという仮説のもと検証を行った．

4.1 RAT のソースコードの調査

File Manager 機能の利用に伴って AccessTime が更新される要因を特定するため，両 RAT の File Manager 機能を実装するソースコードを調査した．検証対象とした AsyncRAT および DCRat は，いずれも GitHub 上で公開されているソースコードを参照した．

調査の結果，AsyncRAT の FileManager.cs では，指定されたフォルダ配下のサブフォルダ一覧およびファイル一覧を取得する処理として，Directory.GetDirectories() および Directory.GetFiles()が利用されていることを確認した．

また，DCRat の FileManager.cs についても調査したところ，同様に Directory.GetDirectories() および Directory.GetFiles()を利用した実装が確認された．

両 RAT において同一の処理が実装されていたことから，これらの API が特徴的な AccessTime 更新挙動に関与している可能性があると考え，個別の動作検証を実施した．

4.2 動作検証の環境

本検証では，C2 通信や外部ネットワークの影響を排除し，コードおよび API の呼び出しが AccessTime に与える影響のみを確認するため，1 台の仮想マシン内で検証する．本検証で使用した環境を以下に示す．

- 環境：仮想マシン 1 台
- OS：Windows 10
- ネットワーク：通信なし

4.3 C#コードでの動作検証

Directory.GetFiles()が AccessTime 更新挙動に与える影響を確認するため，RAT の実装を参考にした検証用プログラムをコード 1 のとおり作成した．

作成したプログラムをコンパイルし，検証用フォルダおよびファイルを対象に実行した．実行前後の MFT タイムスタンプを確認した結果を図 2 に示す．図 2 において，下から 2 行目の 01:19 に確認される AccessTime(Type=".a..")の更新は検証用ファイルの作成に伴うものである．一方，Directory.GetFiles()を含むプログラムを 01:26 に実行した際に AccessTime(Type=".a..") が更新されたのは，「x:/Users/John/Desktop/test」フォルダのみであり，フォルダ内のファイルに対する AccessTime 更新は確認されなかった．

© 2026 Information Processing Society of Japan

```
using System;
using System.IO;

namespace dev
{
 class Program
 {
  static void Main()
  {
   string   path = @"C:¥Users¥john¥Desktop¥test";
   string[] files = Directory.GetFiles(path);

   foreach (string file in files)
   {
    Console.WriteLine(file);
   }
  }
 }
}
```

コード 1　C#による痕跡検証コード
Code1 C# code for verifying traces

```
#include <windows.h>
#include <iostream>
#include <string>

int main()
{
 std::wstring path = L"C:¥¥Users¥¥John¥¥Desktop¥¥
test¥¥*";
 WIN32_FIND_DATAW findData;
 HANDLE    hFind    =    FindFirstFileW(path.c_str(),
&findData);

 do {
  std::wcout << findData.cFileName << std::endl;
 } while(FindNextFileW(hFind, &findData));

 FindClose(hFind);
 return 0;
}
```

コード 2　C++による痕跡検証コード
Code2 C++ code for verifying traces

Date	Type	File Name
2026-07-17T01:16:37Z	macb	x:/Users/john/Desktop/test/test.docx ($FILE_NAME)
2026-07-17T01:16:39Z	..c.	x:/Users/john/Desktop/test/text.txt
2026-07-17T01:18:40Z	.a..	x:/Users/john/Desktop/test/test.docx
2026-07-17T01:19:05Z	.a..	x:/Users/john/Desktop/test/text.txt
2026-07-17T01:19:06Z	.a..	x:/Users/john/Desktop/test/test.zip
2026-07-17T01:19:07Z	.a..	x:/Users/john/Desktop/test/test.xlsx
2026-07-17T01:19:09Z	.a..	x:/Users/john/Desktop/test/test.png
2026-07-17T01:19:10Z	.a..	x:/Users/john/Desktop/test/test.pdf
2026-07-17T01:19:11Z	.a..	x:/Users/john/Desktop/test/test.jpg
2026-07-17T01:26:30Z	.a..	x:/Users/john/Desktop/test

図 2　GetFiles()によるタイムスタンプ更新結果
Figure2 Timestamp update results by GetFiles()

Date	Type	File Name
2026-07-17T01:16:37Z	macb	x:/Users/john/Desktop/test/test.docx ($FILE_NAME)
2026-07-17T01:16:39Z	..c.	x:/Users/john/Desktop/test/text.txt
2026-07-17T01:18:40Z	.a..	x:/Users/john/Desktop/test/test.docx
2026-07-17T01:19:05Z	.a..	x:/Users/john/Desktop/test/text.txt
2026-07-17T01:19:06Z	.a..	x:/Users/john/Desktop/test/test.zip
2026-07-17T01:19:07Z	.a..	x:/Users/john/Desktop/test/test.xlsx
2026-07-17T01:19:09Z	.a..	x:/Users/john/Desktop/test/test.png
2026-07-17T01:19:10Z	.a..	x:/Users/john/Desktop/test/test.pdf
2026-07-17T01:19:11Z	.a..	x:/Users/john/Desktop/test/test.jpg
2026-07-17T01:55:03Z	.a..	x:/Users/john/Desktop/test

図 3　FindFirstFileW()，FindNextFileW()によるタイムスタンプ更新結果
Figure3 Timestamp update results by FindFirstFileW() and FindNextFileW()

この結果から，Directory.GetFiles()単体ではフォルダ内の全ファイルの AccessTime を一斉に更新することはできず，観測された特徴的な痕跡の直接的な原因ではないことが分かった．一方で，引数として指定したフォルダの AccessTime は更新されることを確認した．

さらに，Directory.GetDirectories()および周辺処理で利用されていた GetInfo()についても同様の検証を実施したが，いずれもフォルダ内の全ファイルに対する AccessTime の一斉更新を再現することはできなかった．

4.4　C++(Windows API)コードでの動作検証

Directory.GetFiles()および Directory.GetDirectories()は，内部的に Windows API である FindFirstFile()および FindNextFile()を利用して実装されている．そのため，4.3 節の結果が.NET 固有の実装に起因するものではないことを確認するため，Windows API を直接呼び出す検証を実施した．検証には，FindFirstFileW()および FindNextFileW()を利用してファイル一覧を取得する C++プログラムをコード 2 のとおり作成した．

作成したプログラムを実行し，MFT タイムスタンプへの影響を確認した結果を図 3 に示す．その結果，01:55 にプログラムを実行した際に AccessTime(Type=".a..") が更新されたのは，「x:/Users/John/Desktop/test」フォルダのみであり，フォルダ内ファイルの AccessTime 更新は確認されなかった．

この結果は 4.3 節の結果と一致しており，FindFirstFile()および FindNextFile()によるファイル列挙処理のみでは，AsyncRAT で観測された，フォルダ内の全ファイルの AccessTime がほぼ同時刻に更新される挙動は再現されなかった．

一方，これらの API を実行した際には，対象フォルダの AccessTime が更新されることは確認できた．以上から，フォルダ内の全ファイルの AccessTime 更新との直接的な関連は確認できなかったものの，FindFirstFile(), FindNextFile(), およびこれらを利用する Directory.GetFiles() や Directory.GetDirectories()によってフォルダが参照された場合，少なくとも対象フォルダの AccessTime は更新されることを確認した．

4.5　メソッド、API による痕跡に対する考察

本章の検証は，呼び出されているメソッドや API を確認するだけで AccessTime の更新挙動を推測できるかを検討することを目的としたため，Directory.GetFiles() や FindFirstFile()，FindNextFile()によってファイル一覧を取得する程度の最小限のコードを用いた．

NTFS では，ファイル名はインデックス属性（$I30）としてフォルダ側で管理される．そのため，今回検証したファ

© 2026 Information Processing Society of Japan

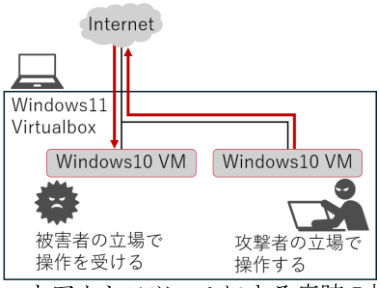


図 4　リモートアクセスツールによる痕跡の検証環境
Figure4 An environment for verifying traces left behind by remote access tools.

イル一覧取得処理ではフォルダの管理データのみが参照され，個々のファイルの AccessTime が更新されなかった可能性がある．一方，AsyncRAT および DCRat ではフォルダ内ファイルの AccessTime が一斉に更新されていることから，実際の File Manager 機能ではファイル属性やデータ領域（$DATA）に関連する情報を取得している可能性がある．

5. サポート詐欺に悪用されるリモートアクセスツールの検証

テクニカルサポート詐欺は現在も活発であり，被害が継続的に報告されている．テクニカルサポート詐欺においても，被害端末からデータが窃取されていないかの調査を求められることがあり，RAT の場合と同様に，探索または窃取されたフォルダおよびファイルの特定が困難であるという課題がある．また，前章までに検証した RAT はいずれも C#で実装されており，開発言語や実装方法が大きく異なる可能性がある正規のリモートアクセスツールにおいて，どのような痕跡が残存するかは明らかではない．

3 章および 4 章の結果から，AccessTime の更新挙動はソフトウェアが取得する情報や内部処理に依存すると考えられる．そこで，開発言語や実装が大きく異なる正規のリモートアクセスツールにおいても，同様の痕跡が確認できるという仮説のもと，AnyDesk および RustDesk を対象に検証した．

5.1 リモートアクセスツールの検証環境

本検証では，実際のリモートアクセスツールの利用形態に近い条件とするため，ローカルに構築した 2 台の仮想マシン間でインターネットを経由した通信を行う図 4 の構成とした．本検証で使用した環境を以下に示す．

- 環境：仮想マシン 2 台
- OS：Windows 10
- ネットワーク：インターネットを経由して 2 台の仮想マシン上のリモートアクセスツールが通信

5.2 AnyDesk の検証結果

AnyDesk の File Manager 機能を介して，フォルダの探索およびファイルの窃取を行い，それぞれでどのような痕跡が残るか検証した．検証の結果，探索したフォルダ自体の AccessTime が更新されることを確認できた．また，ファイルを窃取した場合，窃取したファイル自体の AccessTime が更新されることを確認できた．

先の 2 件の RAT においては，広範囲で AccessTime の更新が行われたのに対し，AnyDesk では実際に操作したフォルダ，ファイルに限り AccessTime が更新されるという違いを確認できた．

5.3 RustDesk の検証結果

RustDesk の File Manager 機能を介して，フォルダの探索およびファイルの窃取を行い，それぞれでどのような痕跡が残るか検証した．検証の結果，探索したフォルダ，ファイルの AccessTime は更新されず，MFT では痕跡が確認できないことが分かった．しかし，ファイルを窃取した場合，窃取したファイル自体の AccessTime が更新されることを確認できた．

検証の際，他の操作による痕跡についても確認していたが，Remote Desktop 機能から Windows 標準のエクスプローラーを介してフォルダを探索した際に，探索したフォルダ内のファイルの AccessTime が更新されることを確認した．

5.4 リモートアクセスツールによる痕跡への考察

AnyDesk および RustDesk は，いずれも RAT と類似の File Manager 機能を備えるが，AccessTime の更新範囲は RAT と同一ではなく，実際に操作したフォルダやファイルに限定される場合や，痕跡が確認できない場合があることを確認した．特に RustDesk は Rust 言語で開発されており，C#で実装された RAT とは異なる痕跡の残り方を示した．このことから，取得する情報が同種であっても，開発言語や実装方法の違いにより，残存する痕跡が異なる場合があると考えられる．

また，RustDesk の Remote Desktop 機能を介して Windows 標準のエクスプローラーを操作した際に，探索したフォルダ内のファイルの AccessTime が一斉に更新された点は，想定していなかった挙動である．ローカル環境でエクスプローラーを直接操作した場合には同様の挙動は確認されていないことから，RustDesk のリモート操作に固有の要因が存在する可能性がある．現時点では要因の特定に至っていないため，今後の課題とする．

6. 検証結果のまとめ

複数のソフトウェアおよびコードを対象とした検証結果を表 2 に示す．本章では，3.1 節で示した以下の評価基準に基づき，痕跡の有効性を整理する．

① ソフトウェアを介した操作時刻に痕跡が残存するか
② 探索した範囲が痕跡で特定可能か

表 2 では，上記の基準に基づき，有効性を〇，△，×で示した．〇は，操作時刻に痕跡が残存し，探索範囲の特定に有効である場合を示す．△は，痕跡は残存するものの，特定できる範囲がフォルダ単位などに限定される場合を示

© 2026 Information Processing Society of Japan

表 2　残存する痕跡の検証結果まとめ
Table2 Summary of Identified Artifacts

対象	検証結果	痕跡の有効性
AsyncRAT の File Manager	探索したフォルダ内のファイルの AccessTime が全て更新される.	○
DCRat の File Manager	探索したフォルダ内のファイルの AccessTime が全て更新される.	○
GetFiles	対象にしたフォルダの AccessTime が更新される.	△
FindFirstFile, FindNextFile	対象にしたフォルダの AccessTime が更新される.	△
AnyDesk の File Manager	探索したフォルダの AccessTime が更新される.	△
RustDesk の File Manager	探索したフォルダの AccessTime は更新されない.	×
RustDesk のリモートデスクトップ	探索したフォルダ内のファイルの AccessTime が全て更新される.	○

す．×は，操作時刻に有効な痕跡が確認できず，探索範囲の特定には利用しにくい場合を示す.

評価の結果，AsyncRAT および DCRat の File Manager では, 操作時刻にフォルダ内ファイルの AccessTime が一斉に更新され, 探索範囲の特定に有効であった. 一方, GetFiles, FindFirstFile，FindNextFile，および AnyDesk の File Manager では，対象フォルダの AccessTime 更新にとどまるため，探索範囲の絞り込みには限定的に有効である. また, RustDesk の File Manager では有効な痕跡は確認できなかったが，Remote Desktop 機能を介した操作では，フォルダ内ファイルの AccessTime が一斉に更新され, 有効な痕跡として利用できる可能性が示された．以上から，痕跡の残り方は実装や操作経路によって異なるものの，操作時刻と更新範囲を対応付けて確認することで，探索または窃取された可能性のある範囲を絞り込む手掛かりとして活用できると考えられる.

7. おわりに

インシデント調査では，攻撃者によって窃取されたファイルを直接特定することは困難であり，ファイルシステムや通信に残された痕跡からその範囲を推定する必要がある. 本研究では，複数のリモートアクセス型マルウェアおよびリモートアクセスツールを対象に検証を行い，ソフトウェアを介してフォルダやファイルを探索した際，参照したフォルダ内のファイルの AccessTime が一斉に更新される場合があることを確認した．一方，Windows 標準のエクスプローラーでは，同様の挙動は確認されなかった．このことから，本研究で確認した AccessTime の更新は，ソフトウェアを介したファイル探索に伴う特徴的な痕跡の一つであると考えられる．インシデント調査において，本痕跡をプログラムの実行痕跡や通信記録と組み合わせて分析することで，攻撃者が探索した可能性のあるフォルダやファイルを抽出し，情報漏洩範囲の絞り込みに活用できる可能性がある．ただし，AccessTime の更新のみでファイルの窃取を判断することはできず，他の痕跡と組み合わせた総合的な分析が必要である.

今後は，検証対象となるソフトウェアを拡大し，AccessTime 更新挙動の共通点の整理と検証結果の精度向上を図るとともに，更新の原因となる Windows API やその呼び出し方についても調査を進めたい.

謝辞　本研究は，国立研究開発法人情報通信研究機構(NICT)による支援を受けて実施した．ここに記して謝意を表する.

参考文献

[1] K. P. Chow, F. Y. W. Law, M. Y. K. Kwan and P. K. Y. Lai, The Rules of Time on NTFS File System, Second International Workshop on Systematic Approaches to Digital Forensic Engineering (SADFE'07), 2007, p. 71-85, doi: 10.1109/SADFE.2007.22.

[2] Michael Galhuber, Robert Luh, Time for Truth: Forensic Analysis of NTFS Timestamps, ARES '21, 2021, no. 44, p. 1-10.

[3] Sebastian Neuner, Artemios G. Voyiatzis, Martin Schmiedecker, Edgar R. Weippl, Timestamp hiccups: Detecting manipulated filesystem timestamps on NTFS, ARES '17, 2017, no. 33, p. 1-6.

[4] B. Carrier and E. Spafford, Getting Physical with the Digital Investigation Process, International Journal of Digital Evidence, Vol. 2, No. 2, 2003, p. 1-21.

[5] Nicole Lang Beebe, Jan Guynes Clark, A hierarchical, objectives-based framework for the digital investigations process, Digital Investigation, Volume 2, Issue 2, 2005, p. 147-167.

[6] Ritzkal, Hendrawan, Ade, Kurniawan, Ridwan, Aprian, Alief, Primasari, Dewi, Subchan, M., Enhancing Cybersecurity Through Live Forensic Investigation of Remote Access Trojan Attacks using FTK Imager Software, International Journal of Safety and Security Engineering. 2024, vol. 14, p. 217-223. 10.18280/ijsse.140121.

© 2026 Information Processing Society of Japan